

Objetivos da Aula:

- entender por que o objeto literal não protege dado, e o que uma classe resolve.
- dominar map e filter, as duas ferramentas que montam o relatório.
- criar as classes Candidato, Eleicao e Usuario, com as regras dentro delas.
- proteger uma tela com middleware e entregar o painel com o relatório de votos.

De onde viemos e para onde vamos

Na Aula 3 nós fizemos:

separamos import e export, e os dados em arquivo próprio.

criamos o Router e dividimos as rotas por assunto.

instalamos o EJS e passamos a devolver página de arquivo, com `res.render`.

reaproveitamos cabeçalho e rodapé com `partials`.

Hoje o assunto é outro:

as páginas já existem, mas qualquer um abre qualquer uma.

e o dado ainda é objeto literal solto: aceita número inválido sem reclamar.

vamos dar regra ao dado com classes, e criar uma porta com fechadura ao painel.

ao final, uma tela de login leva a um painel protegido com o relatório.

Antes de começar: instalando o que a aula precisa

// No terminal, na pasta do projeto:

```
npm install express-session
```

// Saida real:

// added 6 packages, and audited 76 packages in 2s

// O que ja estava instalado das aulas anteriores:

// express 4.22.2 o servidor

// ejss 6.0.1 o motor de views

// nodemon 3.1.14 reinicia sozinho ao salvar

// Conferindo: npm list --depth=0

O roteiro de hoje, em quatro partes

- 1. Classe: dar regra ao dado, para o candidato não nascer errado.
- 2. map e filter: as duas ferramentas que transformam lista em relatório.
- 3. A classe Eleicao: onde moram as regras que envolvem mais de um candidato.
- 4. Login: classe Usuario, sessão, middleware e a tela protegida.

Parte 1: o problema que ficou em aberto

// Em qualquer arquivo, com o objeto literal da Aula 3:

```
const candidato = { id: 1, nome: 'Ana Souza', numero: '10' }
```

```
candidato.numero = 'banana'
```

```
candidato.nmoe = 'Bruno Lima' // era para ser nome
```

```
console.log(candidato)
```

// Saida real do terminal:

```
// { id: 1, nome: 'Ana Souza', numero: 'banana', nmoe: 'Bruno Lima' }
```

```
// nada impediu o numero invalido, e o erro de digitacao
```

```
// criou um campo novo, em silencio.
```

Uma analogia antes de qualquer definição

Pense na diferença entre uma folha em branco e uma ficha de cadastro impressa.

Na folha em branco você escreve o que quiser, do jeito que quiser.

Nada impede escrever o número da urna com letra, ou esquecer o nome.

Na ficha impressa, cada campo já vem com a regra ao lado.

O campo número avisa que são dois dígitos obrigatórios antes de você preencher.

A classe é a ficha impressa.

Ela define de antemão os campos que existem e a regra de cada um. `new Candidato(...)` é preencher uma ficha nova, sempre pela mesma regra.

class, constructor e this

A classe declara o formato

```
class Candidato { ... }
```

o constructor roda toda vez que se cria um novo, com new.

this aponta para o objeto que está sendo criado agora.

Um objeto por vez

new Candidato(...) cria uma instância.
cada instância guarda seus próprios valores, atribuídos com this.

```
candidato instanceof Candidato // true
```

src/dominio/Candidato.js (arquivo novo, completo)

```
// src/dominio/Candidato.js
```

```
// Crie a pasta src/dominio e depois este arquivo.
```

```
export class Candidato {  
  constructor(id, nome, numero) {  
    if (nome.length < 3) {  
      throw new Error('nome precisa ter pelo menos 3 letras')  
    }  
    if (!/^[0-9]{2}$/.test(numero)) {  
      throw new Error('numero precisa ter exatamente dois digitos')  
    }  
  
    this.id = id  
    this.nome = nome  
    this.numero = numero  
  }  
}
```

try e catch (em qualquer arquivo que crie um Candidato)

// throw interrompe na hora. Quem chama precisa se proteger.

```
try {  
  const invalido = new Candidato(2, 'A1', '20')  
  console.log('cadastrado:', invalido)  
} catch (erro) {  
  console.log('falhou:', erro.message)  
}
```

// falhou: nome precisa ter pelo menos 3 letras

// Repare: a variavel invalido nunca chegou a receber nada.

// O objeto invalido nao existe em lugar nenhum do programa.

Onde a gente testa isso? Na pasta scripts

O problema:

a classe não tem tela e não tem rota. Subir o servidor só para ver se ela funciona é caro e lento.

A solução, que já usamos na Aula 2:

um arquivo dentro de scripts/ que importa a classe, exercita ela e imprime o resultado no terminal.

Registrando o comando no package.json:

```
"scripts": {  
  "dev": "nodemon index.js",  
  "candidato:demo": "node scripts/candidatoDemo.js",  
  "array:demo": "node scripts/arrayDemo.js",  
  "eleicao:demo": "node scripts/eleicaoDemo.js"  
}
```

E no terminal:

```
npm run candidato:demo
```

scripts/candidatoDemo.js (arquivo novo, completo)

```
// scripts/candidatoDemo.js  
// Testa a classe Candidato sem precisar subir o servidor.
```

```
import { Candidato } from '../src/dominio/Candidato.js'
```

```
const brutos = [  
  { id: 1, nome: 'Ana Souza', numero: '10' },  
  { id: 2, nome: 'Bruno Lima', numero: '20' },  
  { id: 3, nome: 'Al', numero: '30' },  
  { id: 4, nome: 'Carla Dias', numero: '007' },  
]
```

```
for (const bruto of brutos) {  
  try {  
    const candidato = new Candidato(bruto.id, bruto.nome, bruto.numero)  
    console.log('ok:      ', candidato.numero, candidato.nome)  
  } catch (erro) {  
    console.log('recusado:', bruto.nome, '->', erro.message)  
  }  
}
```

npm run candidato:demo (saída real)

ok: 10 Ana Souza

ok: 20 Bruno Lima

recusado: A1 -> nome precisa ter pelo menos 3 letras

recusado: Carla Dias -> numero precisa ter exatamente dois digitos

Dois passaram, dois foram barrados na porta.

Nenhuma rota, nenhum banco, nenhum navegador envolvido:

a regra e da classe, e ela vale em qualquer lugar.

Erro clássico: esquecer o new

```
const semNew = Candidato(4, "Bruno Lima", "30")
```

```
// Saída real do terminal:
```

```
// TypeError: Class constructor Candidato cannot be  
// invoked without 'new'
```

```
// Correção: toda classe se cria com new na frente.  
const comNew = new Candidato(4, "Bruno Lima", "30")
```

Parte 2: map e filter, com uma analogia

Os dois recebem uma função e devolvem um array novo. A diferença está no que sai.

map é a fábrica de crachás.

Entra uma fila de pessoas, sai uma fila de crachás. A quantidade é a mesma: cada pessoa virou outra coisa.

filter é a catraca com regra.

Entra uma fila de pessoas, sai uma fila de pessoas. Ninguém virou nada: só passou quem atendia à regra, então a fila pode diminuir.

Nenhum dos dois mexe no array original.

Os dois devolvem um array novo, e o de origem continua intacto.

scripts/arrayDemo.js (arquivo novo)

```
// scripts/arrayDemo.js
```

```
const candidatos = [  
  { id: 1, nome: 'Ana Souza', numero: '10' },  
  { id: 2, nome: 'Bruno Lima', numero: '20' },  
  { id: 3, nome: 'Carla Dias', numero: '30' },  
]
```

```
const votos = [  
  { id: 1, candidatoId: 1 },  
  { id: 2, candidatoId: 1 },  
  { id: 3, candidatoId: 2 },  
]
```

```
// map: MESMO tamanho, cada item transformado
```

```
const nomes = candidatos.map((candidato) => candidato.nome)
```

```
// filter: MENOS itens, nenhum transformado
```

```
const votosDaAna = votos.filter((voto) => voto.candidatoId === 1)
```

npm run array:demo (saída real)

1. map: MESMO tamanho, cada item transformado
candidatos.length: 3
nomes.length: 3
nomes: ['Ana Souza', 'Bruno Lima', 'Carla Dias']
2. filter: MENOS itens, nenhum transformado
votos.length: 3
votosDaAna.length: 2
votosDaAna: [{ id: 1, candidatoId: 1 },
 { id: 2, candidatoId: 1 }]
3. o original nunca muda
votos.length depois do filter: 3

Lendo a função de dentro do map, devagar

```
const nomes = candidatos.map((candidato) => candidato.nome)
```

```
// 1. map percorre o array sozinho, sem voce escrever for.
```

```
// 2. a cada volta ele chama a SUA funcao,
```

```
//     passando um item como argumento.
```

```
// 3. o nome candidato e escolha sua: e o parametro.
```

```
// 4. o que a funcao RETORNA vira o item do array novo.
```

```
// A mesma coisa, escrita de forma longa:
```

```
const nomes = candidatos.map(function (candidato) {  
    return candidato.nome  
})
```

```
// A arrow function de uma linha tem return implicito:
```

```
// (candidato) => candidato.nome    ja devolve o nome.
```

map, filter, find e forEach: qual usar (principais métodos de arrays)

map

devolve array do MESMO tamanho, com os itens transformados.

filter

devolve array MENOR ou igual, com os itens que passaram no teste.

A função tem que devolver true ou false.

find

devolve UM item, o primeiro que passar no teste, ou undefined.

Usamos na Aula 3 para achar o candidato pelo número.

forEach

não devolve nada: undefined. Serve só para fazer algo a cada item, como imprimir.

Se você quer um resultado, não é forEach.

O mesmo relatório, do jeito antigo e do jeito novo

```
// Com for, do jeito que já sabíamos:
const relatorio = []
for (const candidato of candidatos) {
  let recebidos = 0
  for (const voto of votos) {
    if (voto.candidatoId === candidato.id) {
      recebidos = recebidos + 1
    }
  }
  relatorio.push({ nome: candidato.nome, votos: recebidos })
}
```

```
// Com map e filter, a mesma coisa em quatro linhas:
const relatorio = candidatos.map((candidato) => {
  const recebidos = votos.filter((v) => v.candidatoId === candidato.id)
  return { nome: candidato.nome, votos: recebidos.length }
})
```

A prova: as duas versões dão o mesmo resultado (saída real)

```
--- com for, do jeito que já sabíamos ---
```

```
[ { nome: 'Ana Souza', votos: 2 },  
  { nome: 'Bruno Lima', votos: 1 },  
  { nome: 'Carla Dias', votos: 0 } ]
```

```
--- com map e filter, a mesma coisa ---
```

```
[ { nome: 'Ana Souza', votos: 2 },  
  { nome: 'Bruno Lima', votos: 1 },  
  { nome: 'Carla Dias', votos: 0 } ]
```

Não é mágica, é o mesmo laço escrito de outro jeito.

Você lê a intenção: "para cada candidato, conte os votos dele".

Parte 3: método, quando a classe também faz coisas

Até aqui a classe só guarda com regra. Mas o objeto também sabe agir sobre os próprios dados.

Método é uma função que mora dentro da classe.

Ela enxerga o `this` daquela instância, então não precisa receber os dados por parâmetro: já os tem.

A classe Eleicao vai guardar as regras do sistema.

Cadastrar um candidato, registrar um voto e apurar o resultado deixam de estar espalhados pelas rotas e passam a ter um lugar só.

src/ dominio/ Eleicao.js (arquivo novo, parte 1 de 2)

```
// src/ dominio/ Eleicao.js
```

```
import { Candidato } from './Candidato.js'
```

```
export class Eleicao {
```

```
  constructor() {
```

```
    this.candidatos = []
```

```
    this.votos = []
```

```
  }
```

```
  cadastrar(nome, numero) {
```

```
    const jaExiste = this.candidatos.find((c) => c.numero === numero)
```

```
    if (jaExiste) {
```

```
      throw new Error(`o numero ${numero} ja esta em uso`)
```

```
    }
```

```
    const id = this.candidatos.length + 1
```

```
    const candidato = new Candidato(id, nome, numero)
```

```
    this.candidatos.push(candidato)
```

```
    return candidato
```

```
  }
```

src/ dominio/ Eleicao.js (parte 2 de 2, continua no mesmo arquivo)

```
  registrarVoto(candidatoId) {
    this.votos.push({ id: this.votos.length + 1, candidatoId })
  }

  apurar() {
    return this.candidatos.map((candidato) => {
      const recebidos = this.votos
        .filter((v) => v.candidatoId === candidato.id)

      return {
        numero: candidato.numero,
        nome: candidato.nome,
        votos: recebidos.length,
      }
    })
  }
}

// Fecha a chave da classe. O arquivo termina aqui.
```

Duas regras, dois donos diferentes

Regra do Candidato:

nome com pelo menos 3 letras.

número com exatamente dois dígitos.

São regras do formato de UM candidato. Ele se valida sozinho, sem olhar para ninguém.

Regra da Eleicao:

não pode existir número repetido.

Essa o candidato não tem como verificar: ele não conhece os outros candidatos.

A pergunta que decide o lugar:

a regra precisa olhar para além de si mesma? Se sim, ela sobe para a Eleicao.

scripts/eleicaoDemo.js (arquivo novo, completo)

```
// scripts/eleicaoDemo.js

import { Eleicao } from '../src/dominio/Eleicao.js'

const eleicao = new Eleicao()

function tentarCadastrar(nome, numero) {
  try {
    eleicao.cadastrar(nome, numero)
    console.log('ok: ', numero, nome)
  } catch (erro) {
    console.log('recusado:', nome, '->', erro.message)
  }
}

tentarCadastrar('Ana Souza', '10')
tentarCadastrar('Bruno Lima', '20')
tentarCadastrar('Carla Dias', '10')

eleicao.registrarVoto(1)
eleicao.registrarVoto(1)
eleicao.registrarVoto(2)

console.table(eleicao.apurar())
```

npm run eleicao:demo (saída real)

ok: 10 Ana Souza

ok: 20 Bruno Lima

recusado: Carla Dias -> o numero 10 ja esta em uso

(index)	numero	nome	votos
0	10	Ana Souza	2
1	20	Bruno Lima	1

console.table desenha a tabela sozinho: otimo para conferir array de objetos no terminal.

Carla Dias nao entrou: o numero 10 ja era da Ana.
Essa e a regra da Eleicao, e nao a do Candidato.

Parte 4: por que o login entra agora

- as páginas da Aula 3 estão todas abertas: qualquer pessoa abriria o cadastro.
- cadastrar candidato e ver o relatório de votos são tarefas do administrador.
- então precisamos de alguém para logar: a classe Usuario.
- e de uma tela de login, que já sabemos fazer, porque a Aula 3 instalou o EJS.
- a votação do público, única por dispositivo ou IP, fica para uma aula mais adiante.

src/dominio/Usuario.js (arquivo novo, completo)

```
// src/dominio/Usuario.js
```

```
export class Usuario {  
  constructor(login, senha) {  
    this.login = login  
    this.senha = senha  
  }  
  
  autenticar(senhaDigitada) {  
    return senhaDigitada === this.senha  
  }  
}
```

```
// Saida real:
```

```
// professor.autenticar('1234') -> true
```

```
// professor.autenticar('0000') -> false
```

HTTP não lembra de ninguém: por isso existe sessão

Cada requisição chega ao servidor sozinha, sem memória do que aconteceu na anterior. O servidor não sabe que aquele navegador acertou a senha há dez segundos.

Sem sessão, o login seria esquecido no instante seguinte.

O usuário acertaria a senha, seria mandado para o painel, e o painel não teria como saber quem ele é.

A sessão é a pulseira da festa.

Na entrada você mostra o documento uma vez e recebe a pulseira. Depois disso, ninguém pede documento de novo: basta a pulseira.

req.session é essa pulseira, e ela existe em toda requisição daquele visitante.

index.js (acrescentar estas duas linhas antes das rotas)

```
// No topo do arquivo:
```

```
import session from 'express-session'
```

```
// Depois do express.static que ja existe, e ANTES do app.use das rotas:
```

```
app.use(express.urlencoded({ extended: true })))
```

```
app.use(session({  
  secret: 'xxx333-333xxx-0000',  
  resave: false,  
  saveUninitialized: false,  
}))
```

```
// urlencoded -> faz o req.body existir quando vier formulario
```

```
// session -> faz o req.session existir em toda requisicao
```

Middleware: o que é, de verdade

Middleware é uma função que roda no meio do caminho, entre a chegada da requisição e a rota que vai respondê-la.

Ela recebe sempre os mesmos três parâmetros: req, res e next.

E tem duas saídas possíveis: chamar next(), que significa "pode seguir", ou responder ela mesma, o que encerra a requisição ali e a rota nunca roda.

Se a sessão é a pulseira, o middleware é a portaria.

A portaria olha a pulseira. Tem pulseira, next(), pode entrar. Não tem, ela mesma manda de volta para a entrada, e você não chega a ver a festa.

Você já usou três middlewares sem saber

No index.js, cada `app.use` é um middleware:

`express.static` serve arquivo de `public/`

`express.urlencoded` monta o `req.body`

`session` monta o `req.session`

Todos rodam ANTES das suas rotas, em toda requisição.

Por isso a ordem importa:

se o `session` vier DEPOIS das rotas, `req.session` ainda não existe quando a rota roda, e o login quebra.

A ordem do nosso `index.js`:

`static`, `urlencoded`, `session`, rotas.

src/middlewares/exigirLogin.js (arquivo novo, completo)

```
// src/middlewares/exigirLogin.js  
// Crie a pasta src/middlewares e depois este arquivo.
```

```
export function exigirLogin(req, res, next) {  
  if (!req.session.usuario) {  
    return res.redirect('/login')  
  }  
  
  next()  
}
```

```
// Sem pulseira: responde com um redirect e ENCERRA.  
// O return na frente do redirect e o que impede  
// o código de continuar e chamar next() também.
```

```
// Com pulseira: next() passa a vez para a rota.
```

Onde o exigirLogin entra: no meio da rota

```
// Em src/routes/painelRoutes.js:
```

```
import { exigirLogin } from '../middlewares/exigirLogin.js'
```

```
// Rota SEM protecao, como as da Aula 3:
```

```
router.get('/login', (req, res) => { ... })
```

```
//           ^caminho   ^funcao que responde
```

```
// Rota COM protecao: o middleware vira o 2o argumento,
```

```
// entre o caminho e a funcao que responde.
```

```
router.get('/painel', exigirLogin, (req, res) => { ... })
```

```
//           ^caminho   ^porteiro   ^funcao que responde
```

```
// Pode ter mais de um, separados por virgula.
```

```
// Eles rodam na ordem em que aparecem.
```

views/login.ejs (arquivo novo, completo)

```
<%- include('partials/cabecalho') %>
```

```
<% if (erro) { %>
```

```
  <p class="erro"><%= erro %></p>
```

```
<% } %>
```

```
<form method="POST" action="/login">
```

```
  <label>Login: <input type="text" name="login"></label>
```

```
  <label>Senha: <input type="password" name="senha"></label>
```

```
  <button type="submit">entrar</button>
```

```
</form>
```

```
<%- include('partials/rodape') %>
```

```
// method="POST" manda os dados no corpo, e nao na URL.
```

```
// action e o caminho da rota que vai receber.
```

```
// o name de cada input vira a chave dentro do req.body.
```

Sem o `express.urlencoded`, o formulário não chega

// Se voce esquecer aquele `app.use` no `index.js`:

```
router.post('/login', (req, res) => {  
  const { login, senha } = req.body
```

// Saida real do terminal:

// `req.body` = undefined

// `TypeError: Cannot destructure property "login" of`
// `'req.body'` as it is undefined.

// Com o middleware no lugar:

// `req.body` = { login: 'professor', senha: '1234' }

src/routes/painelRoutes.js (arquivo novo, parte 1 de 2)

```
import { Router } from 'express'
import { Usuario } from '../dominio/Usuario.js'
import { eleicao } from '../dados/eleicao.js'
import { exigirLogin } from '../middlewares/exigirLogin.js'

const router = Router()
const professor = new Usuario('professor', '1234')

router.get('/login', (req, res) => {
  res.render('login', { titulo: 'Entrar', erro: null })
})

router.post('/login', (req, res) => {
  const { login, senha } = req.body

  if (login !== professor.login || !professor.autenticar(senha)) {
    return res.render('login', {
      titulo: 'Entrar', erro: 'login ou senha invalidos' })
  }

  req.session.usuario = professor.login // entrega a pulseira
  res.redirect('/painel')
})
```

src/routes/painelRoutes.js (parte 2 de 2, mesmo arquivo)

```
router.get('/painel', exigirLogin, (req, res) => {
  res.render('painel', {
    titulo: 'Painel',
    usuario: req.session.usuario,
    relatorio: eleicao.apurar(),
  })
})

router.post('/painel/candidatos', exigirLogin, (req, res) => {
  const { nome, numero } = req.body

  try {
    eleicao.cadastrar(nome, numero)
  } catch (erro) {
    console.log('cadastro recusado:', erro.message)
  }

  res.redirect('/painel')
})

router.get('/logout', (req, res) => {
  req.session.destroy(() => res.redirect('/login'))
})

export default router
```

views/painel.ejs (arquivo novo, completo)

```
<%- include('partials/cabecalho') %>

<p>Bem vindo, <%= usuario %>.</p>

<h2>Cadastrar candidato</h2>
<form method="POST" action="/painel/candidatos">
  <input type="text" name="nome" placeholder="nome">
  <input type="text" name="numero" placeholder="numero">
  <button type="submit">cadastrar</button>
</form>

<h2>Relatorio de votos</h2>
<table>
  <% for (const linha of relatorio) { %>
    <tr>
      <td><%= linha.numero %></td>
      <td><%= linha.nome %></td>
      <td><%= linha.votos %> voto(s)</td>
    </tr>
  <% } %>
</table>

<a href="/logout">sair</a>

<%- include('partials/rodape') %>
```

index.js (arquivo completo: a ordem é o que faz funcionar)

```
import express from 'express'
import session from 'express-session'
import path from 'node:path'
import painelRoutes from './src/routes/painelRoutes.js'

const app = express()

app.set('view engine', 'ejs')
app.set('views', path.resolve('views'))

app.use(express.static('public'))
app.use(express.urlencoded({ extended: true }))
app.use(session({ secret: 'aula4', resave: false,
  saveUninitialized: false }))

app.use(painelRoutes)

app.listen(3000, () => {
  console.log('Servidor rodando em http://127.0.0.1:3000')
})
```

O fluxo inteiro funcionando (saída real)

```
GET /painel sem login  
302 -> /login
```

```
POST /login senha errada  
a pagina volta com: login ou senha invalidos
```

```
POST /login senha certa  
302 -> /painel
```

```
GET /painel com a sessao  
Bem vindo, professor.  
10 Ana Souza 2 voto(s)  
20 Bruno Lima 1 voto(s)
```

```
POST /painel/candidatos nome=Carla Dias numero=30  
302 -> /painel, e a tabela agora tem:  
30 Carla Dias 0 voto(s)
```

Atenção: o que aqui é didático e não vai para produção

A senha está em texto puro dentro do código.

Funciona para aprender classe e login, mas em projeto real a senha se guarda com hash, nunca legível, e o usuário vem do banco de dados.

Os dados vivem na memória do servidor.

Reiniciou o servidor, tudo volta ao estado inicial. É exatamente esse incômodo que justifica a próxima etapa do curso.

O secret da sessão está escrito no arquivo.

O lugar dele é uma variável de ambiente, no .env.

Mãos à obra: a estrutura de hoje

Domínio, middleware e rotas:

src/dominio/Candidato.js

src/dominio/Usuario.js

src/dominio/Eleicao.js

src/middlewares/exigirLogin.js

src/routes/painelRoutes.js

Scripts de teste:

scripts/candidatoDemo.js

scripts/arrayDemo.js

scripts/eleicaoDemo.js

Novas telas:

views/login.ejs

views/painel.ejs

Arquivo alterado:

index.js (urlencoded, session e a rota nova)

Nova dependência:

npm install express-session

Para praticar

- Crie `src/dominio/Voto.js` exigindo `candidatoId`, faça a `Eleicao` usar `new Voto`, e teste em `scripts/votoDemo.js`.
- Use `map` para gerar a lista de números de urna já ocupados, e `filter` para listar só os candidatos sem nenhum voto.
- Mostre o erro na tela: em vez de `console.log` no `catch`, renderize o painel passando a mensagem de erro.
- Crie um segundo middleware, `registrarAcesso`, que só imprime a hora e a URL, e coloque antes do `exigirLogin`.
- Tente abrir `/painel` numa janela anônima e confirme que o middleware devolve para o login.

Referências

MDN Web Docs. Classes, constructor, this, Array.prototype.map, Array.prototype.filter, try...catch.

<https://developer.mozilla.org/pt-BR/>

Express.js. Using middleware e Writing middleware.

<https://expressjs.com/en/guide/using-middleware.html>

express-session. Documentação do pacote. <https://www.npmjs.com/package/express-session>

PINHO, Diego Martins de. ECMAScript 6: entre de cabeça no futuro do JavaScript. Casa do Código, 2017.

PEREIRA, Caio Ribeiro. Livro de NodeJS. Casa do Código, 2013. Atualizado em 09/2021.